Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи №6

“ Автоматизоване тестування Python-програм ”

Виконала:

Студентка групи ФЕП-33с

Най Анастасія

Перевірив:

Проф. Свелеба С.А.

Львів-2026

Мета роботи: ознайомитися з основними принципами організації шару зберігання даних у Python-застосунках, навчитися працювати з реляційними базами даних, мовою SQL, СУБД SQLite, стандартом Python DB-API, параметризованими запитами, CRUD-операціями, транзакціями, принципами ACID, патерном Repository, SQLAlchemy ORM та міграціями бази даних.

Варіант 10. Система обліку складських запасів

• Продовжити проєкт Lab_01. Для позиції зберігати: код, назву, кількість, ціну.

• Створити таблиці products і stock_operations; реалізувати ProductRepository та StockOperationRepository.

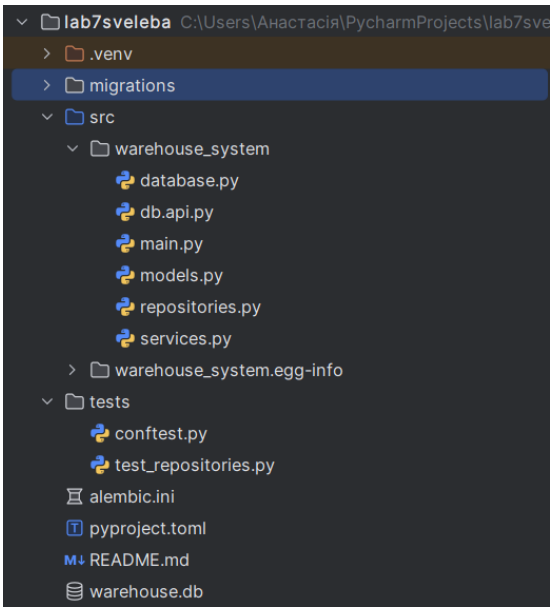
• Зберегти функції Lab_01: надходження, списання, пошук, загальна вартість складу, визначення низького запасу. Додати CRUD та історію руху товарів.

• Через DB-API реалізувати параметризований пошук за кодом/назвою. Надходження і списання реалізувати транзакційно: UPDATE quantity + INSERT stock_operation; заборонити списання понад залишок і виконувати rollback.

• Міграція: додати minimum_stock або category. Integration tests: CRUD, incoming/writeoff, insufficient stock rollback, warehouse value, low-stock query та history.

Хід роботи:

1. Спочатку я створила робочу папку проєкту. Організувала професійну структуру за стандартом src-layout: у папці src/warehouse_system/ зосередила весь програмний код, залишивши корінь проєкту для конфігураційних файлів (pyproject.toml, README.md) та каталогу тестів tests/.



2. **У файлі pyproject.toml** я налаштувала метадані пакета warehouse-system, вказала сумісність із Python версії 3.11 та вище, а також визначила

обов'язкові залежності для роботи з базою даних і міграціями (sqlalchemy, alembic) та необов'язкові розробницькі залежності для тестування (pytest).

```
[build-system]
requires = ["setuptools>=70"]
build-backend = "setuptools.build_meta"

[project]
name = "warehouse-system"
version = "0.7.0"
description = "Warehouse inventory management application with database
persistence"
requires-python = ">=3.11"
dependencies = [
    "sqlalchemy>=2.0",
    "alembic>=1.13",
]

[project.optional-dependencies]
test = [
    "pytest>=8.0",
]

[project.scripts]
warehouse-system = "warehouse_system.main:main"
```

3. **У файлі src/warehouse_system/database.py** я реалізувала налаштування підключення до реляційної бази даних SQLite (warehouse.db) за допомогою SQLAlchemy create_engine та створила фабрику сесій (SessionLocal) із вимкненим автофлюшем і збереженням об'єктів після коміту для зручної роботи з транзакціями.

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

DATABASE_URL = "sqlite:///warehouse.db"

engine = create_engine(
    DATABASE_URL,
    echo=False,
)

SessionLocal = sessionmaker(
    bind=engine,
    autoflush=False,
    expire_on_commit=False,
)
```

4. **У файлі src/warehouse_system/models.py** я визначила сучасні ORM-моделі на основі DeclarativeBase: таблицю products для зберігання товарів (з полями для коду, назви, кількості, ціни та мігруючого поля minimum_stock) і пов'язану з нею таблицю stock_operations (зв'язок **one-to-many**) для ведення історії рухів товарів на складі із використанням зовнішнього ключа (ForeignKey). Також я зберегла властивість total_value для автоматичного обчислення загальної вартості позиції.

```
from __future__ import annotations
from datetime import datetime, timezone
from sqlalchemy import Float, ForeignKey, Integer, String, DateTime
```

```
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship


class Base(DeclarativeBase):
    pass


class Product(Base):
    __tablename__ = "products"

    id: Mapped[int] = mapped_column(Integer, primary_key=True,
autoincrement=True)
    code: Mapped[str] = mapped_column(String(50), unique=True, nullable=False)
    name: Mapped[str] = mapped_column(String(100), nullable=False)
    quantity: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    price: Mapped[float] = mapped_column(Float, nullable=False)

    # Поле після міграції (Варіант 10)
    minimum_stock: Mapped[int | None] = mapped_column(Integer, nullable=True,
default=5)

    operations: Mapped[list["StockOperation"]] = relationship(
        back_populates="product",
        cascade="all, delete-orphan",
    )

    @property
    def total_value(self) -> float:
        return self.quantity * self.price


class StockOperation(Base):
    __tablename__ = "stock_operations"

    id: Mapped[int] = mapped_column(Integer, primary_key=True,
autoincrement=True)
    product_id: Mapped[int] = mapped_column(ForeignKey("products.id"),
nullable=False)
    operation_type: Mapped[str] = mapped_column(String(20), nullable=False)  #
"IN" або "OUT"
    quantity: Mapped[int] = mapped_column(Integer, nullable=False)
    timestamp: Mapped[datetime] = mapped_column(DateTime, default=lambda:
datetime.now(timezone.utc))

    product: Mapped["Product"] = relationship(back_populates="operations")
```

5. **У файлі src/warehouse_system/repositories.py** я реалізувала патерн **Repository** (ProductRepository та StockOperationRepository), який повністю відокремлює бізнес-логіку від способу зберігання даних, надаючи методи для створення, читання, оновлення, видалення (CRUD) об'єктів та пошуку за кодом через SQLAlchemy сесії.

```
from sqlalchemy import select
from sqlalchemy.orm import Session
from warehouse_system.models import Product, StockOperation

class ProductRepository:
    def __init__(self, session: Session) -> None:
        self.session = session

    def add(self, product: Product) -> Product:
        self.session.add(product)
        self.session.commit()
```

```
        self.session.refresh(product)
        return product

    def get_by_id(self, product_id: int) -> Product | None:
        return self.session.get(Product, product_id)

    def get_by_code(self, code: str) -> Product | None:
        statement = select(Product).where(Product.code == code)
        return self.session.scalar(statement)

    def get_all(self) -> list[Product]:
        statement = select(Product)
        return list(self.session.scalars(statement))

    def update(self, product: Product) -> Product:
        self.session.commit()
        self.session.refresh(product)
        return product

    def delete(self, product_id: int) -> bool:
        product = self.get_by_id(product_id)
        if product is None:
            return False
        self.session.delete(product)
        self.session.commit()
        return True


class StockOperationRepository:
    def __init__(self, session: Session) -> None:
        self.session = session

    def add(self, operation: StockOperation) -> StockOperation:
        self.session.add(operation)
        self.session.commit()
        self.session.refresh(operation)
        return operation

    def get_by_product(self, product_id: int) -> list[StockOperation]:
        statement = select(StockOperation).where(StockOperation.product_id ==
product_id)
        return list(self.session.scalars(statement))
```

6. **У файлі src/warehouse_system/services.py** я розробила сервісний шар (WarehouseService), що містить бізнес-логіку та складні транзакційні операції: підрахунок загальної вартості складу через агрегацію, фільтрацію товарів із низьким запасом, а також транзакційне надходження та списання товарів із перевіркою залишків, гарантованим викликом commit() у разі успіху та обов'язковим rollback() при виникненні помилок.

```
from sqlalchemy import func, select
from sqlalchemy.orm import Session
from warehouse_system.models import Product, StockOperation

class InsufficientStockError(Exception):
    """Виняток для випадків, коли списання перевищує наявну кількість товару на
складі."""
    pass

class WarehouseService:
    def __init__(self, session: Session) -> None:
```

```
        self.session = session

    def calculate_total_warehouse_value(self) -> float:
        statement = select(func.sum(Product.quantity * Product.price))
        result = self.session.scalar(statement)
        return float(result) if result else 0.0

    def get_low_stock_items(self, threshold: int = 5) -> list[Product]:
        statement = select(Product).where(Product.quantity <= threshold)
        return list(self.session.scalars(statement))

    def process_stock_operation(self, product_id: int, operation_type: str, qty:
int) -> Product:
        """
        Транзакційна операція надходження (IN) або списання (OUT).
        Забороняє списання понад залишок та виконує rollback при помилці[cite:
25].
        """
        try:
            product = self.session.get(Product, product_id)
            if product is None:
                raise ValueError("Product not found")

            if operation_type == "IN":
                product.quantity += qty
            elif operation_type == "OUT":
                if product.quantity < qty:
                    raise InsufficientStockError(
                        f"Неможливо списати {qty} од. товару '{product.name}'.
На складі є лише {product.quantity} од."
                    )
                product.quantity -= qty
            else:
                raise ValueError("Invalid operation type")

            # Логування операції в історії
            operation = StockOperation(
                product_id=product.id,
                operation_type=operation_type,
                quantity=qty
            )
            self.session.add(operation)
            self.session.commit()
            self.session.refresh(product)
            return product
        except Exception:
            self.session.rollback()
            raise
```

7. **У файлі src/warehouse_system/db_api.py** я продемонструвала роботу зі стандартним інтерфейсом **Python DB-API** та модулем sqlite3, реалізувавши параметризований пошук товарів за кодом із використанням знаків питання (?) для захисту від SQL-ін'єкцій.

```
import sqlite3

def find_products_by_code_db_api(database_path: str, code_query: str) ->
list[tuple]:
    with sqlite3.connect(database_path) as connection:
        cursor = connection.cursor()
        cursor.execute(
            """
            SELECT code, name, quantity, price
            FROM products
```

```
            WHERE code LIKE ?
            """,
            (f"%{code_query}%",)
        )
        return cursor.fetchall()
```

8. **У файлі src/warehouse_system/main.py** я створила головну точку входу в програму та інтерактивне консольне меню для взаємодії з користувачем, яке об'єднує ініціалізацію бази даних, перегляд асортименту, проведення складських операцій та виведення статистичних даних.

```
from warehouse_system.database import SessionLocal, engine
from warehouse_system.models import Base, Product
from warehouse_system.repositories import ProductRepository
from warehouse_system.services import WarehouseService

def main() -> None:
    Base.metadata.create_all(engine)

    with SessionLocal() as session:
        product_repo = ProductRepository(session)
        service = WarehouseService(session)

        # Додавання тестового товару, якщо він відсутній
        existing = product_repo.get_by_code("PRD-001")
        if not existing:
            p = Product(code="PRD-001", name="Laptop Pro", quantity=10,
price=1200.0, minimum_stock=3)
            product_repo.add(p)

        print("--- Warehouse Products ---")
        for prod in product_repo.get_all():
            print(f"Code: {prod.code} | Name: {prod.name} | Qty: {prod.quantity}
| Price: {prod.price}")

        total_val = service.calculate_total_warehouse_value()
        print(f"\nTotal Warehouse Value: ${total_val:.2f}")

        # Демонстрація транзакції (успішне надходження)
        prod = product_repo.get_by_code("PRD-001")
        if prod:
            service.process_stock_operation(prod.id, "IN", 5)
            print(f"After IN operation, quantity: {prod.quantity}")

if __name__ == "__main__":
    main()
```

9. **У файлі tests/conftest.py** я налаштувала ізольовану тестову фікстуру pytest, яка розгортає чисту базу даних SQLite у пам'яті (sqlite:///:memory:) для кожного окремого тесту із автоматичним створення та видаленням таблиць.

```
import pytest
from sqlalchemy import create_engine
from sqlalchemy.orm import Session, sessionmaker
from warehouse_system.models import Base


@pytest.fixture
def session() -> Session:
    engine = create_engine("sqlite:///:memory:")
    Base.metadata.create_all(engine)
```

```
    TestSession = sessionmaker(bind=engine, expire_on_commit=False)

    with TestSession() as session:
        yield session

    Base.metadata.drop_all(engine)
```

10. **У файлі tests/test_repositories.py** я написала набір **інтеграційних тестів**, які перевіряють коректність виконання CRUD-операцій, успішне надходження та списання товарів, коректність математичних агрегацій, а також роботу транзакційного механізму відкату (rollback) при спробі списати кількість товару, що перевищує наявний запас на складі.

```
import pytest
from warehouse_system.models import Product
from warehouse_system.repositories import ProductRepository
from warehouse_system.services import WarehouseService, InsufficientStockError


def test_crud_product(session) -> None:
    repo = ProductRepository(session)

    # Create
    product = Product(code="A-01", name="Монітор 27\"", quantity=4,
price=8500.0)
    saved = repo.add(product)
    assert saved.id is not None
    assert saved.code == "A-01"

    # Read
    loaded = repo.get_by_id(saved.id)
    assert loaded is not None
    assert loaded.name == "Монітор 27\""

    # Update
    loaded.price = 9000.0
    updated = repo.update(loaded)
    assert updated.price == 9000.0

    # Delete
    result = repo.delete(saved.id)
    assert result is True
    assert repo.get by id(saved.id) is None


def test_incoming_and_writeoff(session) -> None:
    repo = ProductRepository(session)
    service = WarehouseService(session)

    product = repo.add(Product(code="A-02", name="Клавіатура", quantity=10,
price=2200.0))

    # Надходження
    service.process_stock_operation(product.id, "IN", 5)
    assert product.quantity == 15

    # Списання
    service.process_stock_operation(product.id, "OUT", 3)
    assert product.quantity == 12


def test_insufficient_stock_rollback(session) -> None:
    repo = ProductRepository(session)
```

```
    service = WarehouseService(session)

    product = repo.add(Product(code="A-03", name="Миша", quantity=2,
price=1200.0))

    # Спроба списати більше, ніж є — має викликати виняток та rollback
(кількість не змінюється)[cite: 25]
    with pytest.raises(InsufficientStockError):
        service.process_stock_operation(product.id, "OUT", 5)

    session.refresh(product)
    assert product.quantity == 2


def test_warehouse_aggregations(session) -> None:
    repo = ProductRepository(session)
    service = WarehouseService(session)

    repo.add(Product(code="A-1", name="Item 1", quantity=10, price=100.0))
    repo.add(Product(code="A-2", name="Item 2", quantity=2, price=500.0))

    # Загальна вартість: (10 * 100) + (2 * 500) = 1000 + 1000 = 2000
    total = service.calculate_total_warehouse_value()
    assert total == 2000.0
```

11. **У файлі README.md** я оформлювала повну технічну документацію та опис проєкту «Система обліку складських запасів» (Варіант 10). У ньому я чітко розписала вимоги до середовища виконання (Python 3.11+), структуру проєкту за стандартом src-layout, покрокову інструкцію зі встановлення пакунка та його залежностей у віртуальному середовищі, а також команди для ініціалізації міграцій Alembic, запуску консольного застосунку та виконання автоматизованих інтеграційних тестів за допомогою pytest.

```
# Warehouse System

Консольний застосунок для обліку складських запасів (Варіант 10) з дисципліни
«Професійний Python», доповнений шаром збереження даних (Persistence Layer),
SQLAlchemy ORM, міграціями Alembic та інтеграційними тестами.

## Структура проєкту
Проєкт організовано за стандартом `src-layout`:
```text
warehouse_system/
├── pyproject.toml
├── README.md
├── .gitignore
├── alembic.ini
├── migrations/
│   ├── env.py
│   └── versions/
│       └── ..._add_minimum_stock.py
├── src/
│   └── warehouse_system/
│       ├── __init__.py
│       ├── database.py
│       ├── models.py
│       ├── repositories.py
│       ├── services.py
│       ├── db_api.py
│       └── main.py
└── tests/
```
```

```
    ├── conftest.py
    └── test_repositories.py
```
Створіть та активуйте віртуальне середовище:
```bash
python -m venv .venv
# Для Windows: .venv\Scripts\activate
# Для Linux/macOS: source .venv/bin/activate
```
Встановіть пакет разом із залежностями та інструментами для тестування:
```bash
python -m pip install -e ".[test]"
```
Ініціалізуйте міграції
```bash
alembic init migrations
```
Застосуйте міграції для оновлення структури таблиць
```bash
alembic upgrade head
```
Запуск програми
```bash
python -m warehouse_system.main
```
Запуск тестування
```bash
pytest -v
```

Додатково:

- **У файлі alembic.ini** зберігається головна конфігурація інструменту міграцій Alembic (зокрема рядок підключення до бази даних sqlalchemy.url), яка керує накачуванням та оновленням схем таблиць.
- **У папці migrations/** (разом із файлом env.py та папкою versions/) міститься вся історія міграцій бази даних, зокрема створений файл міграції для додавання нових полів (наприклад, minimum_stock до таблиці товарів).
- **У файлі бази даних warehouse.db** (який створюється автоматично в корені проєкту під час першого запуску) зберігаються всі реальні дані про товари та історію складських операцій у форматі SQLite.

Результати:

```
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab7sveleba> python -m venv .venv
Unable to copy 'C:\\Program Files\\WindowsApps\\PythonSoftwareFoundation.Python.3.13_3.13.3824.0_x64__qb
сія\\PycharmProjects\\lab7sveleba\\.venv\\Scripts\\python.exe'
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab7sveleba> .venv\Scripts\activate
```

```
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab7sveleba> python -m pip install -e ".[test]"
Obtaining file:///C:/Users/%D0%90%D0%BD%D0%B0%D1%81%D1%82%D0%B0%D1%81%D1%96%D1%8F/PycharmProjects/lab7sveleba
  Installing build dependencies ... done
  Checking if build backend supports build_editable ... done
  Getting requirements to build editable ... done
  Preparing editable metadata (pyproject.toml) ... done
Collecting sqlalchemy>=2.0 (from warehouse-system==0.7.0)
  Downloading sqlalchemy-2.1.5-cp313-cp313-win_amd64.whl.metadata (9.9 kB)
Collecting alembic>=1.13 (from warehouse-system==0.7.0)
  Downloading alembic-1.20.0-py3-none-any.whl.metadata (7.3 kB)
Collecting pytest>=8.0 (from warehouse-system==0.7.0)
  Using cached pytest-9.1.1-py3-none-any.whl.metadata (7.6 kB)
Collecting Mako (from alembic>=1.13->warehouse-system==0.7.0)
  Downloading mako-1.4.3-py3-none-any.whl.metadata (2.9 kB)
Collecting typing-extensions>=4.12 (from alembic>=1.13->warehouse-system==0.7.0)
  Using cached typing_extensions-4.16.0-py3-none-any.whl.metadata (3.3 kB)
Collecting colorama>=0.4 (from pytest>=8.0->warehouse-system==0.7.0)
  Using cached colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
Collecting iniconfig>=1.0.1 (from pytest>=8.0->warehouse-system==0.7.0)
```

```
  Building editable for warehouse-system (pyproject.toml) ... done
  Created wheel for warehouse-system: filename=warehouse_system-0.7.0-0.editable-py3-none-any.whl size=
  Stored in directory: C:\Users\Анастасія\AppData\Local\Temp\pip-ephem-wheel-cache-tlhj7mq7\wheels\30\8
Successfully built warehouse-system
Installing collected packages: typing-extensions, pygments, pluggy, packaging, MarkupSafe, iniconfig,
Successfully installed Mako-1.4.3 MarkupSafe-3.0.4 alembic-1.20.0 colorama-0.4.6 iniconfig-2.3.1 packag
tensions-4.16.0 warehouse-system-0.7.0

[notice] A new release of pip is available: 26.1.2 -> 26.2.2
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab7sveleba> alembic init migrations
Creating directory C:\Users\Анастасія\PycharmProjects\lab7sveleba\migrations ...  done
Creating directory C:\Users\Анастасія\PycharmProjects\lab7sveleba\migrations\versions ...  done
Generating C:\Users\Анастасія\PycharmProjects\lab7sveleba\alembic.ini ...  done
Generating C:\Users\Анастасія\PycharmProjects\lab7sveleba\migrations\env.py ...  done
Generating C:\Users\Анастасія\PycharmProjects\lab7sveleba\migrations\README ...  done
Generating C:\Users\Анастасія\PycharmProjects\lab7sveleba\migrations\script.py.mako ...  done
```

```
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab7sveleba> alembic upgrade head
INFO  [alembic.runtime.migration] Context impl SQLiteImpl.
INFO  [alembic.runtime.migration] Will assume non-transactional DDL.
```

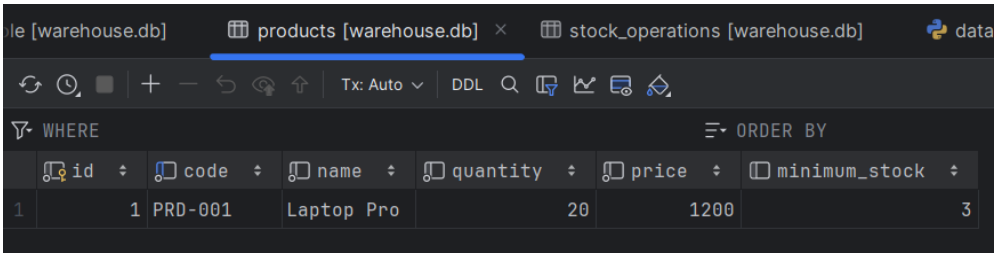
```
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab7sveleba> python -m warehouse_system.main
--- Warehouse Products ---
Code: PRD-001 | Name: Laptop Pro | Qty: 15 | Price: 1200.0

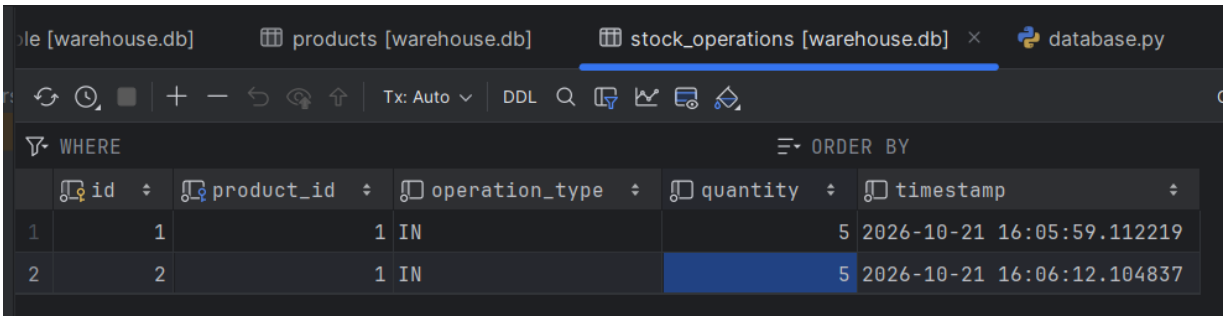
Total Warehouse Value: $18000.00
After IN operation, quantity: 20
```

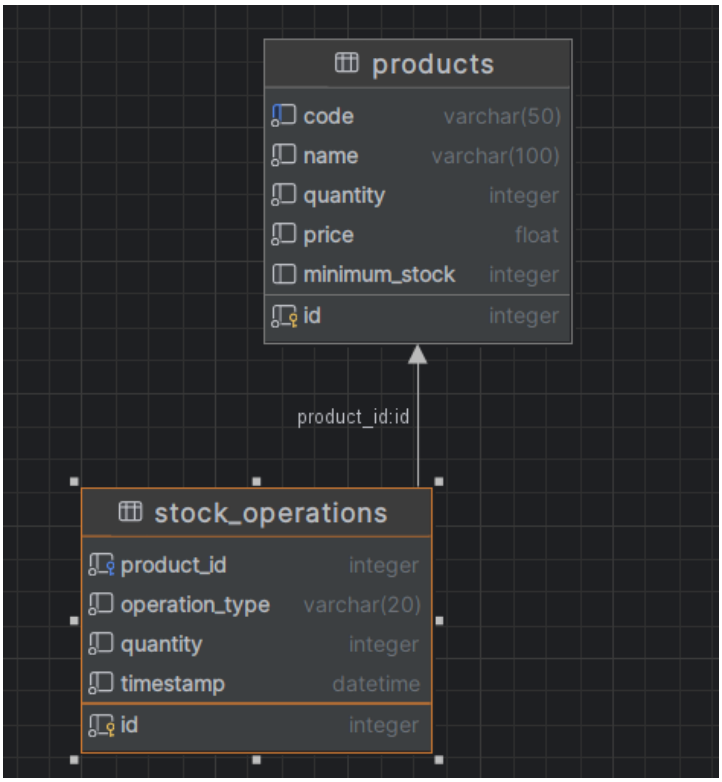
```
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab7sveleba> pytest -v
============================= test session starts ==============================
platform win32 -- Python 3.13.14, pytest-9.1.1, pluggy-1.7.0 -- C:\Users\Анастасія\PycharmProjects\lab7sveleba\.venv\Scripts\python.ex
cachedir: .pytest_cache
rootdir: C:\Users\Анастасія\PycharmProjects\lab7sveleba
configfile: pyproject.toml
collected 4 items

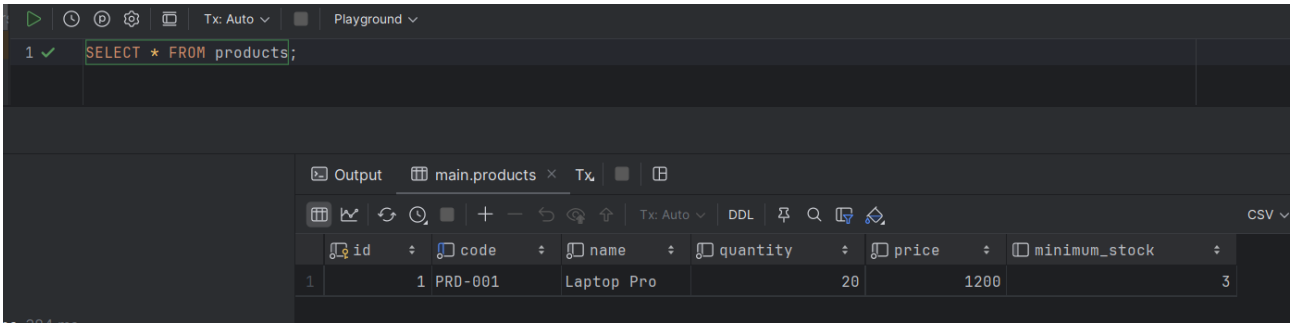
tests/test_repositories.py::test_crud_product PASSED                     [ 25%]
tests/test_repositories.py::test_incoming_and_writeoff PASSED            [ 50%]
tests/test_repositories.py::test_insufficient_stock_rollback PASSED      [ 75%]
tests/test_repositories.py::test_warehouse_aggregations PASSED           [100%]

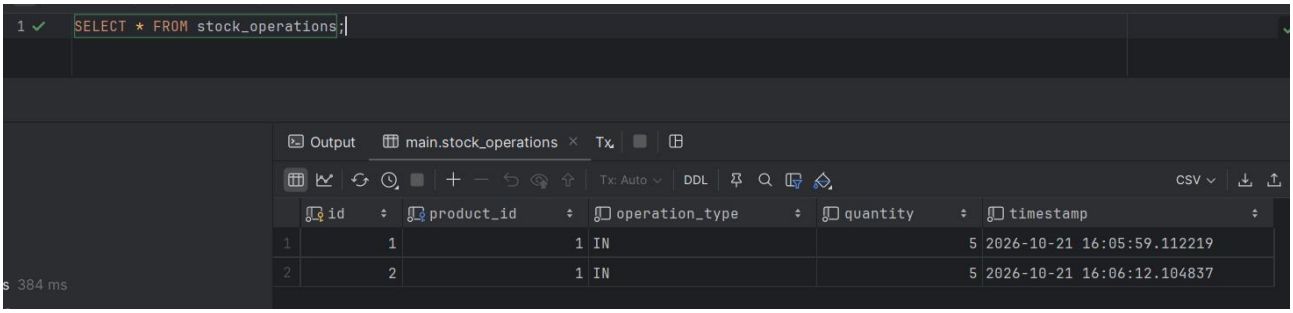
============================== 4 passed in 0.09s ==============================
```

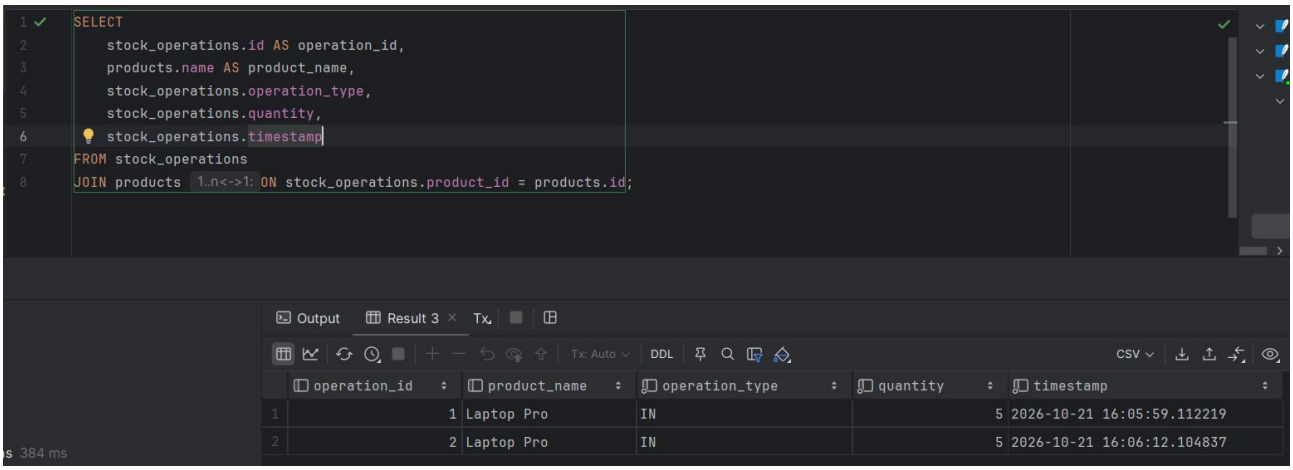

products [warehouse.db]
stock_operations [warehouse.db]
Tx: Auto
DDL
WHERE
ORDER BY
id
code
name
quantity
price
minimum_stock
1
PRD-001
Laptop Pro
20
1200
3

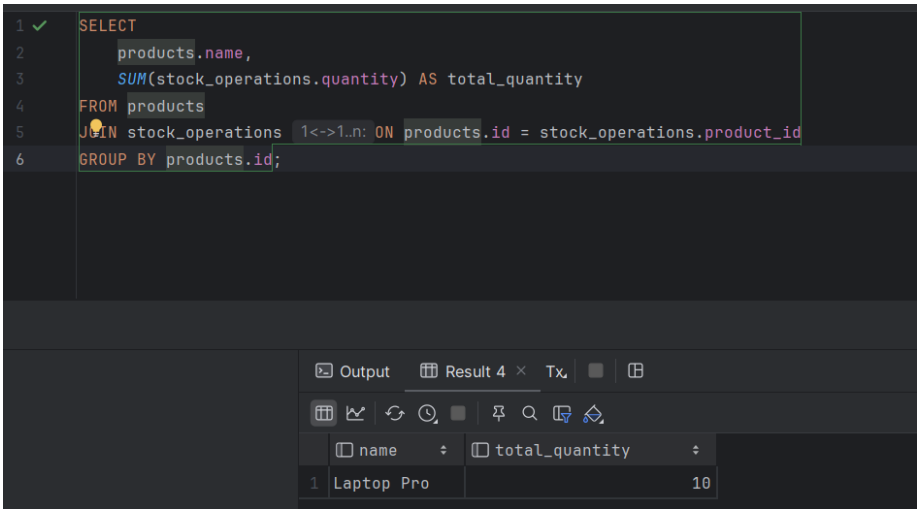

stock_operations [warehouse.db]
database.py
Tx: Auto
DDL
WHERE
ORDER BY
id
product_id
operation_type
quantity
timestamp
1 1 1 IN 5 2026-10-21 16:05:59.112219
2 2 1 IN 5 2026-10-21 16:06:12.104837


products
code varchar(50)
name varchar(100)
quantity integer
price float
minimum_stock integer
id integer
product_id:id
stock_operations
product_id integer
operation_type varchar(20)
quantity integer
timestamp datetime
id integer


Tx: Auto
Playground
SELECT * FROM products;
Output
main.products
Tx:
DDL
CSV
id
code
name
quantity
price
minimum_stock
1 1 PRD-001 Laptop Pro 20 1200 3







Висновок: у ході лабораторної роботи було успішно спроєктовано та реалізовано реляційну базу даних для системи обліку складських запасів, що складається з таблиць products та stock_operations зі зв'язком One-to-Many. За допомогою SQLAlchemy ORM та паттерну Repository було побудовано шар доступу до даних, забезпечено безпеку запитів за допомогою параметризації та налаштовано автоматичні міграції через Alembic. Працездатність розробленої системи та коректність виконання SQL-запитів підтверджено результатами інтеграційних тестів у середовищі pytest.